APPENDIX 1

Platform Differences

Although Docker strives for complete platform independence, there's at least one notable way in which the platforms differ. Let's take a quick look and see what the deal is.

File Ownership and Permissions

There are some subtle differences when it comes to how file ownership and permissions are handled on the different Docker platforms (Mac, Linux, and Windows). These differences can cause a slight issue when you write files to a mounted volume from inside a container.

The issue stems from the fact that there are usually different user accounts being used inside and outside a container. Typically, you use your normal user account on your machine, but inside a container, the default user is the container's `root` account. This means that files created inside the container are owned by that `root` user.

When you exit the container, the question is, will you be able to modify these files that are owned by `root`? The answer is slightly different for each platform.

On Docker for Windows, files created inside the container have file permissions that allow them to be modified by anyone (the equivalent of Unix file permission `777`). This means there's no issue with using and modifying the files outside the container.

Docker for Mac uses its own separate file-sharing system called `osxfs`.[1] It does some trickery[2] to make it seem like the mounted files are owned by whichever

1. https://docs.docker.com/docker-for-mac/osxfs/
2. https://stackoverflow.com/questions/43097341/docker-on-macosx-does-not-translate-file-ownership-correctly-in-volumes